

Horizontal Pod Autoscaling in Kubernetes Cluster using Long Short-Term Memory

Abstract. Containerization is a major trend in software development as an alternative to virtualization. It involves the encapsulation of software code, libraries, packages, and dependencies so that it can run uniformly and consistently on any infrastructure. On a single server or virtual machine by using the same operating system we can deploy multiple applications with the help of containerization. One such open-source container orchestration technology that supports declarative setup and automation for managing containerized workloads and services is Kubernetes. Horizontal pod autoscaling plays a pivotal role in the modern-day world due to diversified user requests and resources. This paper proposes a custom auto scaler to perform horizontal pod autoscaling using LSTM. The effectiveness of this LSTM-based auto scaler is compared with the default auto scaler based on the response times and SMAPE (Symmetric Mean Absolute Percentage Error) error value. Results reveal that the LSTM model performs better in predicting the scaling of pods compared to the ARIMA model.

Keywords: Kubernetes · Pod Autoscaling · Response time · HorizontalPodAutoscaler · Faster Custom Autoscaler · LSTM · ARIMA

1 Introduction

Containerization allows to deploy multiple applications on a single virtual machine with same operating system. For this reason containerizing application is preferred over virtualization. In this case, Kubernetes is used as orchestration tool for scaling pods. Master-Slave architecture of Kubernetes supports scalability by allowing user to add worker nodes. This helps in reducing the work load on available worker nodes. Master node will load the work to available worker nodes in case any worker node dies. In Comparison to other orchestration tools, Kubernetes setup is more scalable and network policies are implemented by network providers such as Calico or Cilium. It also offers pod security policies, RBAC authorization and secrets management. In this work Kuberetes is used as the container orchestration tool. The objective is to build a custom HorizontalPodAutoscaler that performs scaling of workloads to match the user's demand. Horizontal pod autoscaler increases the pods if the workload increases and if the load decreases, HorizontalPodAutoscaler instruct the resources to scale down the pods.

In kubernetes two kinds of autoscaling is performed Horizontal and Vertical autoscaling. Both autoscalers used in modifying the resources. Horizontal Pod Autoscaler Scales number of pods to handle computational workload. HPA

keeps track of resources such as CPU or Memory usage or other custom metrics generated by any metric server. It increases and decreases the resources by modifying the number of replicas of pods according to workload request. If the workload is more than the threshold set, then pods are created and pods are released if the request made is less than the threshold value. This scaling procedure will continue till the replicas number matches the threshold. If we want to use custom metrics to define rules on how the HPA handles scaling your pods, cluster needs to be linked to a time-series database holding the metrics we want to use. In case of custom metrics, the metrics should be stored in a time-series database and that should be linked to the cluster. In Vertical pod autoscaling, resources in each node is modified. That may include CPU, memory or disk usage of node. VPA creates pool of node on the basis of workload request. It will kill the pods which don't have the recommended resource configuration and the controller is used to recreate the pods with updated configuration. In this paper we are using LSTM and ARIMA models and LSTM model gives less SMAPE (Symmetric Mean Absolute Percentage Error) error value than ARIMA model. So, the proposed faster custom autoscaler performs better than the default HPA, as we predict the scaling of pods using LSTM.

The organisation of this paper is as follows: Section II contains the effort on implementing different methods for autoscaling in Kubernetes. Section III includes the algorithm of each module along with the proposed system design. section IV brings forth the results of proposed work and its differentiation with the different Machine Learning algorithms. and the last section includes the conclusions and future work.

2 Background Study

2.1 Related work

In [3] the authors explain Auto-scaling as a Defence Application in Kubernetes: The paper introduces ATHENA and shows how to construct a cloud-native application with orchestration on the Cloud using Docker and Kubernetes. Five NECTAR Research Cloud virtual machines and one Microsoft Azure experimental node made up the ATHENA Kubernetes cluster. HPA successfully spawns more compute resources when we raise the workload (i.e., the number of repeats and the duration of the simulation) (Pods). As we add more resources, the pace at which runtime increases decreases; this is a definite sign that auto-scaling has worked. By default, the metrics are synchronised once every 30 seconds, and scaling up or down is only permitted if there hasn't been rescaling in the previous three to five minutes. The HPA avoids the hasty implementation of decisions that clash in this way (oscillation).

The author provides information on the Improved Q Network Auto-Scaling Microservice in [4] . Building design To meet shifting demands, a cloud service provider needs flexible resource management techniques, such as provisioning and auto-scaling. Workload-based automatic scaling is a technique that is utilised in both open-source and commercial computing platforms. It grows the instances

by increasing the volume of incoming requests. The algorithm’s capacity to handle more requests than are configured to be processed simultaneously is called concurrency. However, because a number of variables can influence throughput and latency depending on the workloads and complexity of the infrastructure features, it has been challenging to identify.

In [5] the author proposes a thorough approach to container resource management with three distinct goals. The first is to effectively schedule containers on available resources in order to optimize their initial location. The second is to perform resource autoscaling based on the workload of the cluster at the moment. The third one is a rescheduling technique that, when feasible, consolidates applications into fewer VMs to further promote resource efficiency. Experiments show that controlling the cluster size and application placement dynamically can result in significant cost reductions. When compared to the default Kubernetes scheduler, we find that our suggested alternatives can cut the cost by 58%. The authors in [6] propose auto-scaling strategy to solve the response delay problem. The proposed method is composition of empirical mode decomposition and ARIMA models. This helps in predicting the load of the pod. Results of this method prove that Experiment proves that the capacity expansion before the peak load and helps in reduction of the response time.

In [7], by dynamically adjusting the amount of resource units, or "pods," HPA enables uninterrupted service without the need to restart the entire system. In order to provide important information about HPA’s operational characteristics, we explore it through a variety of studies in this paper. Autoscaling is one of the most crucial components of Kubernetes since it enables the resilient operation of containerized applications and services without the need for manual intervention. Kubernetes has three different categories of autoscalers. By varying the number of execution and resource units, known as pods, HorizontalPodAutoscaler (HPA) offers high availability. In response to a trigger, HPA generates new pods to share workloads without interfering with those already running inside the cluster. Technically speaking, HPA is a control loop that kube-controller-manager has put in place. By default, kube-controller-manager analyzes the gathered metrics against their thresholds set in configurations of HPA once every 15 seconds, or "sync cycle."

In this study [8], we provide a Double Threshold Horizontal Pod Autoscaler (DHPA) algorithm that classifies event scale into three categories: scale-out, no scale, and scale-in. And last, for each of the three scenarios, we use two thresholds that are further separated into no scaling (anti jitter), normal scaling, and fast scaling. The DHPA algorithm’s expansion and contraction operations are in a state of dynamic equilibrium. The number of pods dispatched by the method deviates significantly from the theoretical calculation because it is placed within the message queue as a single job each time it is scaled, but the overall equilibrium is dynamic. The initial HPA method counts the number of pods present across the entire cluster.

2.2 Prediction Model

The Fig 1 shows the input and outputs of an LSTM for a single timestep. Since LSTMs maintain an internal state to keep track of data that has already been seen, they are appropriate for modeling sequence data. Input, forget and Output are the three gates of the LSTM. LSTM enables to store information in both directions, either backward or forwards. and these gates are a method of learning used by LSTMs. The gated cell architecture of lstm keeps the memory of the terms that are uncovered in earlier time steps. Helps in making more educated predictions based on long historical data.

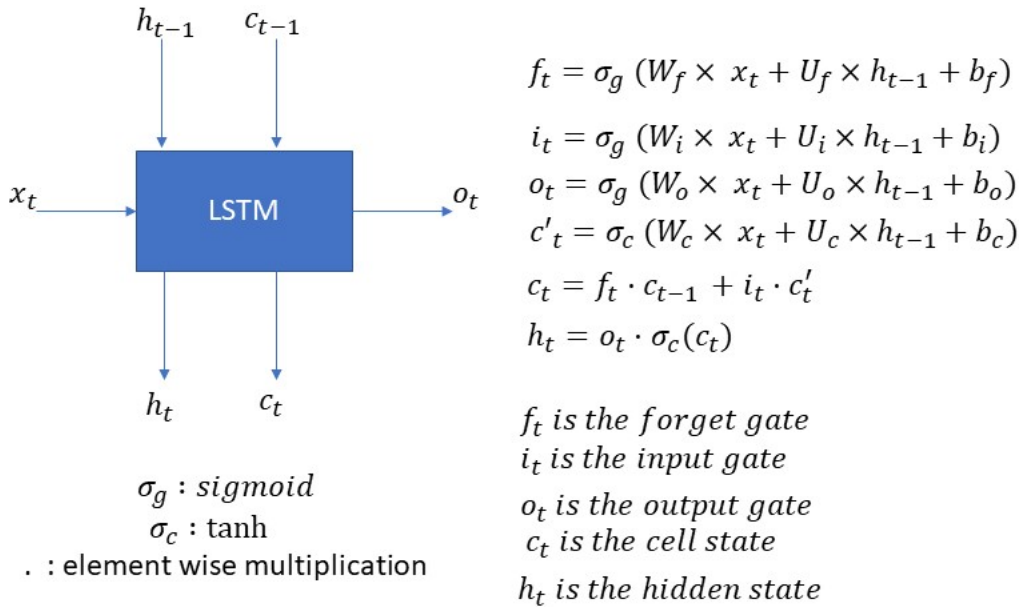


Fig. 1. Structure of LSTM and the corresponding equations for a single timestep.

3 Proposed System

Kubernetes cluster consists of worker nodes and one master node to monitor the worker nodes. Worker node hosts pods, in which containers are present. The clusters comprises of many control planes. The Control plane is responsible for maintaining a record of all kubernetes objects and works to achieve desired state of the system objects. These system objects consists of three main logical components i.e, Controller manager, watches the state of the cluster and is responsible for joining pods to services. The API Server is used by the client to manage their requests. It acts as the gateway to the cluster so it is accessible to the clients from outside the cluster. The worker nodes contains two main services, kubelet and kube-proxy. kubelet is an agent runs on each node. The job of kubelet is to make sure that containers in pods are running healthy. To achieve network communication to the pods, network rules are used. These network rules are maintained by Kube-proxy.

System Model defines the objects of a system. That include, components, resources and services. These elements are described and represented using architecture and system design. Kubernetes has master-slave architecture. It describes the components of by Control plane (master) and worker nodes. System diagram of this work explains the interactions between the client and and API server of the master and behaviour of the system towards the user's request.

Custom metric scraping is done using metrics.k8s.io API. Prometheus is used to fetch this custom metrics i.e, CPU usage and response time of pods. Prometheus adapter is used to serve the metrics out of prometheus.

Horizontal pod autoscaler scales the number of pods in replica set considering the custom metrics. The System diagram explains the interactions between the client and the Kubernetes cluster. The client will interact with cluster through API server. Client can request for pod creation or allocation of node using API server of control plane. 'etcd' database will be having all the information regarding the available workloads and stores the data about the communications between the pods. Horizontal pod autoscaler will perform the scaling of pods based on user's demand.

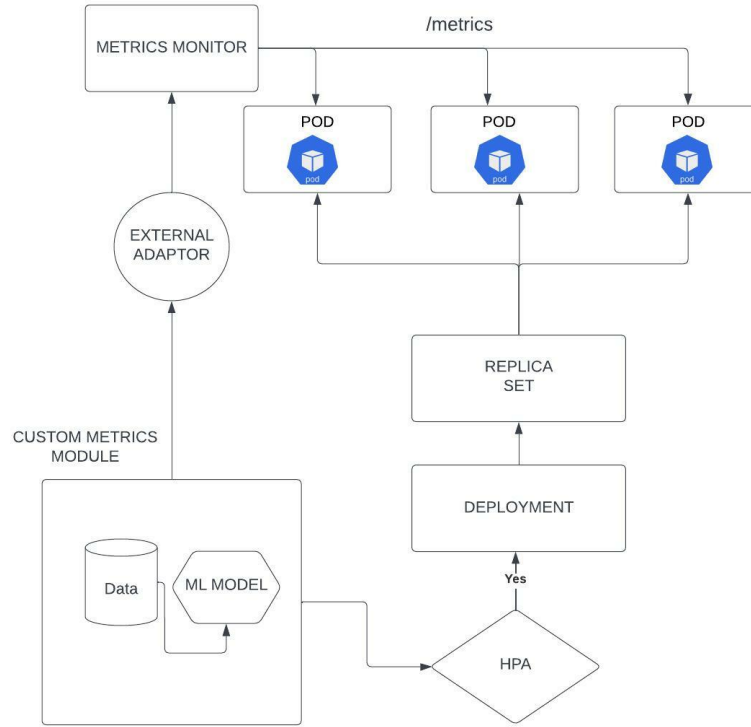


Fig. 2. System Diagram.

Horizontal Pod Autoscaler Scales number of pods to handle computational workload. HPA keeps track of resources such as CPU or Memory usage or other custom metrics generated by any metric server. If the workload is more than the threshold set, then pods are created and pods are released if the request made is less than the threshold value. This scaling procedure will continue till the replicas number matches the threshold.

We create a test thread and scaling the workloads. Workloads will be scaled up if the request for deployment is more than the threshold i.e, max replicas based on response time and CPU usage. In order to perform efficient utilization of resources, pods are released if the requirement is minimum.

Algorithm 1 shows the Horizontal Pod AutoScaling in Kubernetes.

Algorithm 1 Horizontal Pod AutoScaling Algorithm using LSTM

Input: $U_{target}, ActivePods$
 $\triangleright$ Target value and Active Pods
Output: P $\triangleright$ Target number of Pods to deploy
while true **do**
 for all $i \in ActivePods$ **do**
 $U_i = getRelativeMetrics(i);$
 $U = U \cup U_i$
 end for
 $P = ceil(sum(u)/u_{target})$
 $wait(t)$ $\triangleright$ wait t seconds, the control loop period
end while

4 Result and Discussion

In this section results of proposed methodology are discussed. This section includes experimental setup and result analysis of machine learning models LSTM and ARIMA.

4.1 Experimental setup

The Kubernetes cluster is set up with master node(16 gb) and numerous other slave nodes. In this work Ubuntu 22.04 live server is used and Kubernetes cluster set up is created of version 1.23. Docker is used as container runtime. To perform horizontal pod autoscaling, custom metrics are accessed from the pods, using prometheus monitoring tool (2.24 version). HAProxy of 2.2 stable LTS is used. In this work LSTM model is used to predict the scaling of pods by creating the replicas of pod based on the workload.

4.2 Result Analysis

Custom metrics are taken into time-series database, and model is trained to predict the scaling up or down of pods. The dataset includes HTTP requests, the response time and the status of the requests. LSTM and ARIMA models are compared and to evaluate the models, Mean square and SMAPE values are considered. The fig. 3 shows the training results of ARIMA model.

The fig. 4 shows the testing results of ARIMA model. It is observed that on testing with new requests default hpa generates more replicas than the requirement compared to custom. Custom hpa performs efficient scaling by generating pods according to threshold and requirements.

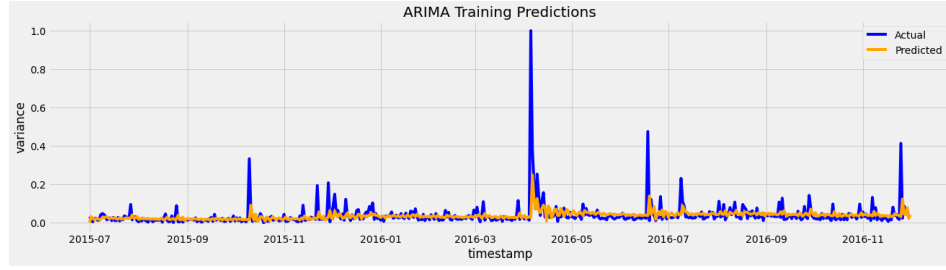


Fig. 3. Training Performance of Arima Model.

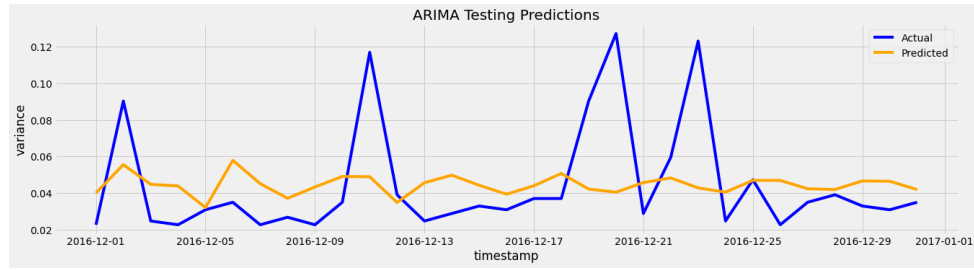


Fig. 4. Testing Performance of Arima Model.

Fig. 6 presents the actual and predicted scaling of the pods based on workload request. Response time and scaling is made on the basis of requests made on different days. LSTM model is trained on the HTTP requests on different days. It is observed that default hpa will increase the replicas of pods even when the workload request is less.

On testing with new requests made on few days, we can see custom hpa creating required replicas and releasing the resources when the workload is decreased. But default hpa creates more replicas of pods. So, LSTM model performs better compared to ARIMA model and gives less error on testing with the new HTTP requests.

Performance analysis of Default and custom HPA is made considering the latency / response time for HTTP requests. From fig. 7 it is observed that default HPA reacts much slower to the relative change in the number of requests or time since the last request (idle time) than the proposed HPA.

In contrast, proposed custom autoscaler performs better compared to default horizontal pod autoscaler. Faster custom HPA speeds up the scaling up/down of pods by predicting the workload ahead of default HPA. In order to handle workload in the near future, custom HPA removes part of replicas if the workload decreases.

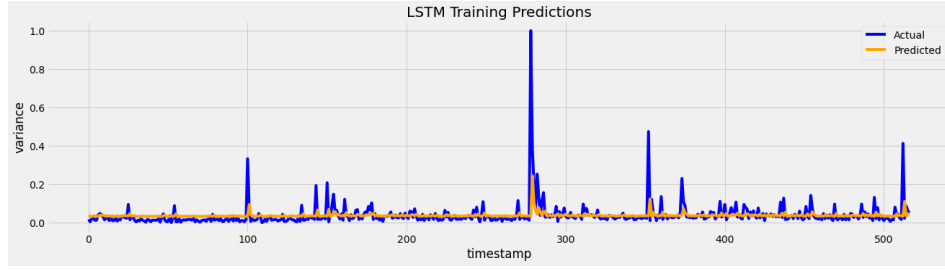


Fig. 5. Training Performance of LSTM Model.

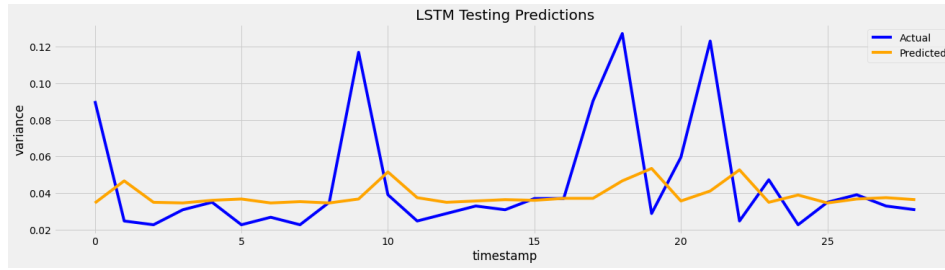


Fig. 6. Testing Performance of LSTM Model.

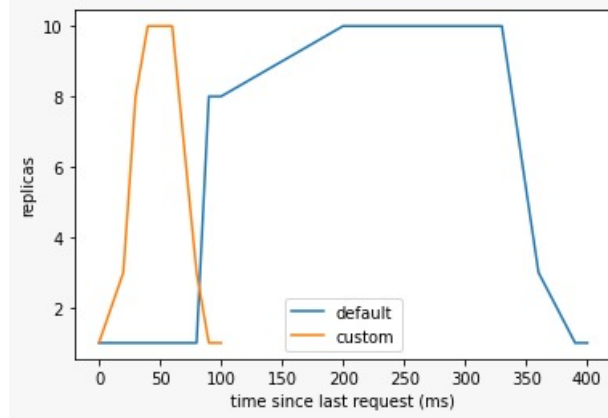


Fig. 7. Performance of Default and Custom HPA.

Table 1 describes the SMAPE error values of machine learning models LSTM and ARIMA. We can observe that lstm model gives less SMAPE error value on testing with new requests, than arima model. So, we can consider the lstm model to predict the scaling of pods ahead of default hpa.

Table 1. SMAPE values of LSTM and ARIMA Model

	LSTM Model	ARIMA Model
Training error (SMAPE)	31.02	55.68
Testing error (SMAPE)	10.99	43.44

5 Conclusion

In this work, we designed custom autoscaler which is more effective than the default horizontal pod autoscaler of Kubernetes. We set up the Kubernetes cluster and then scraped the metrics using Prometheus. To access the metrics outside the cluster, prometheus adapter is used. We consider response time as metric for auto scaling. The autoscaler in the cluster was trained using LSTM to predict the replicas of pods. Same procedure was followed for ARIMA to predict the scaling. The SMAPE error value of LSTM was less than the ARIMA model. So, we modeled an LSTM based autoscaler and compared the average response times of the LSTM based autoscaler with the default HPA. As future work, we plan to consider additional metrics in the design of auto scaler.